ONLINE STORE
DATA WAREHOUSE

Using SQL Server and SQL Server Integration Services (SSIS)

Data Engineering Bootcamp – Final Project

Prepared by: Eman Mohamed Sobhy

Source Database → SSIS ETL → Data Warehouse → SQL Reports

1. Introduction

This project implements a small data warehouse for an online store using Microsoft SQL Server and SQL Server Integration Services (SSIS). The solution separates operational source data from an analytical structure so that management can review completed sales, compare product performance, and understand customer purchasing patterns.

The workflow includes creating and populating a relational source database, designing a star-schema data warehouse, transferring and transforming data through an SSIS package, and validating the loaded data with SQL queries.

1.1 Project Objectives

- Design relational tables and enforce primary-key and foreign-key relationships.
- Use SQL joins, grouping, aggregate functions, filtering, and ordering to answer business questions.
- Create dimension and fact tables using a star-schema design.
- Implement an Extract, Transform, Load (ETL) workflow in SSIS.
- Validate row counts, business rules, referential integrity, and sales totals.

2. Source Database Design

The operational database is named MiniStoreSource. It contains four related tables representing customers, products, orders, and order lines.

Table	Purpose	Important relationships
Customers	Stores customer identifiers, names, and cities.	CustomerID is the primary key.
Products	Stores product names, categories, and list prices.	ProductID is the primary key.
Orders	Stores the customer, date, and status for each order.	CustomerID references Customers.
OrderItems	Stores the products, quantities, and actual unit prices within each order.	OrderID references Orders; ProductID references Products.

The supplied sample-data script inserts 7 customers, 6 products, 10 orders, and 18 order lines. It includes both Completed and Cancelled orders, a customer with no orders, and a customer whose only order is cancelled. Several customer names contain extra leading or trailing spaces to test the trimming transformation.

Sales amount is calculated as Quantity × UnitPrice. UnitPrice represents the amount charged on the order line and may differ from the current product price, as demonstrated by a discounted order line.

2.1 Source SQL Queries

The source queries answer the required questions: list customers and cities; show orders with customer names and dates; calculate sales by order, product, and customer; identify the highest-revenue product; and count orders by status. Completed-sales reports filter to OrderStatus = 'Completed' so cancelled orders do not contribute to sales revenue.

A bonus query uses HAVING to identify customers whose completed sales exceed 10,000.

3. Data Warehouse Design

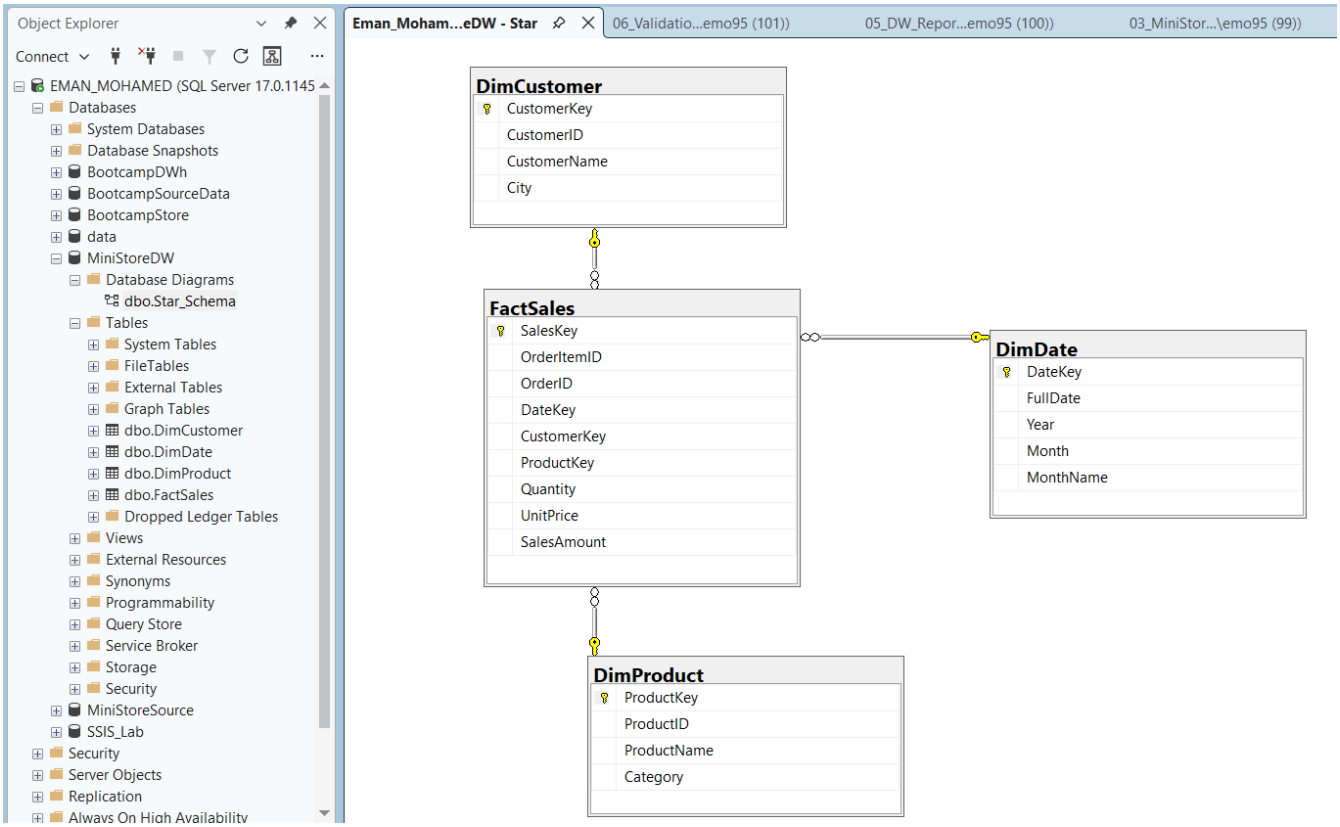
The analytical database is named MiniStoreDW. It uses a star schema with three dimension tables surrounding one sales fact table.

Table	Role and contents
DimCustomer	Customer dimension; contains CustomerKey (surrogate key), source CustomerID, CustomerName, and City.
DimProduct	Product dimension; contains ProductKey (surrogate key), source ProductID, ProductName, and Category.
DimDate	Date dimension; contains DateKey in YYYYMMDD format, FullDate, Year, Month, and MonthName.
FactSales	Sales fact table; contains order-line identifiers, dimension keys, Quantity, UnitPrice, and SalesAmount.

3.1 Star Schema and Fact Grain

The grain of FactSales is one row per source order line (one OrderItems record). An order containing multiple product lines therefore creates multiple fact rows. SalesKey is an identity-based warehouse key, while OrderItemID identifies the original source line and is constrained to be unique in the fact table.

FactSales has foreign keys to DimCustomer, DimProduct, and DimDate. The warehouse uses surrogate dimension keys (CustomerKey and ProductKey) rather than storing source identifiers as the dimension relationships. This makes the fact table suitable for grouped analysis by customer, product, date, and city.



4. ETL Implementation with SSIS

The SSIS solution is named MiniStoreETL and the package is named LoadMiniStoreDW.dtsx. The package extracts source data from MiniStoreSource, applies the required transformations and key mappings, and loads the warehouse tables.

4.1 ETL Workflow

Order	Load step	Main transformation / rule
1	DimCustomer	Read source customers and trim unnecessary leading/trailing spaces from CustomerName.
2	DimProduct	Read source products and load product attributes.
3	DimDate	Load distinct order dates; derive DateKey (YYYYMMDD), Year, Month, and MonthName.
4	FactSales	Join Orders and OrderItems; keep Completed orders; look up CustomerKey, ProductKey, and DateKey; calculate Quantity × UnitPrice.

Dimension tables are loaded before FactSales so that every fact row can reference an existing dimension record. Cancelled orders are excluded from FactSales because the warehouse is intended to support completed-sales analysis. A full load is sufficient for this beginner project; incremental loading is outside the project scope.

4.2 SSIS Package Evidence

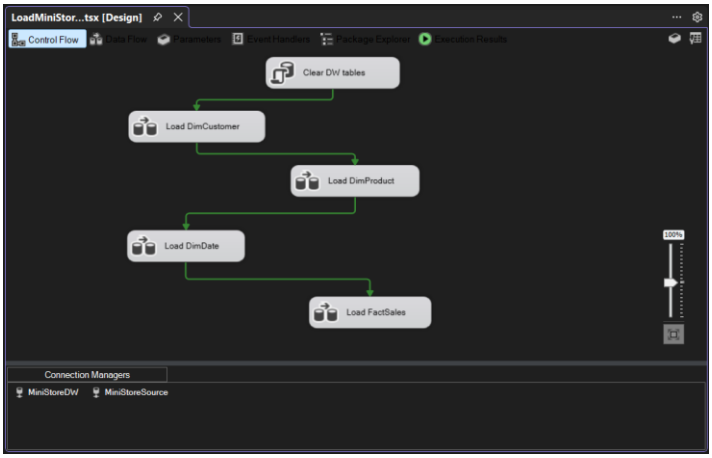


Figure 1: SSIS Control Flow

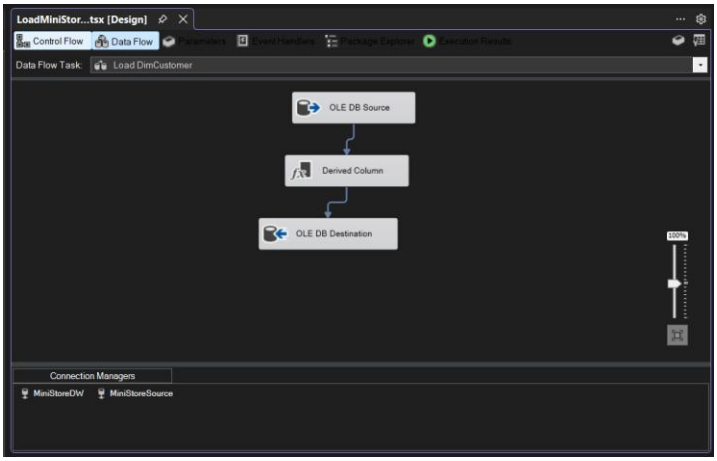


Figure 2: SSIS Data Flow (Load DimCustomer)

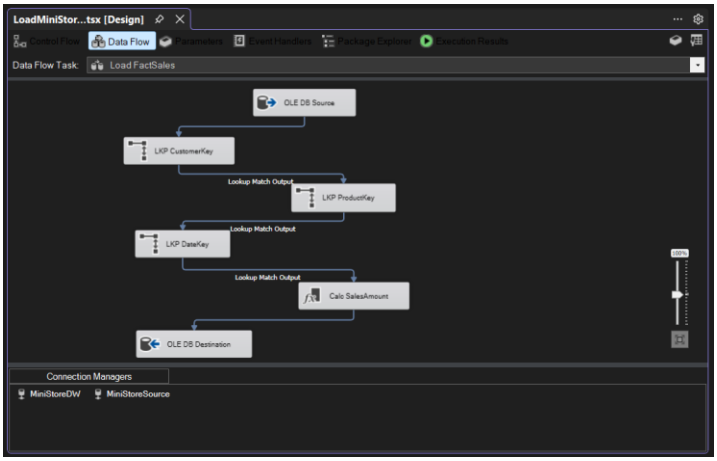


Figure 3: SSIS Data Flow (Load FastSales)

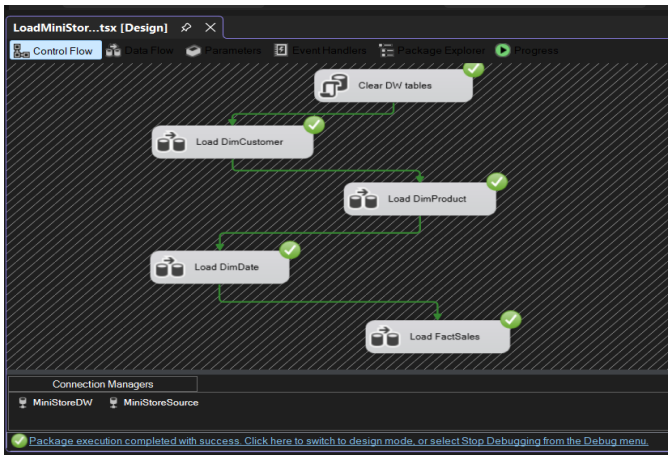


Figure 4: SUCCESSFUL PACKAGE EXECUTION

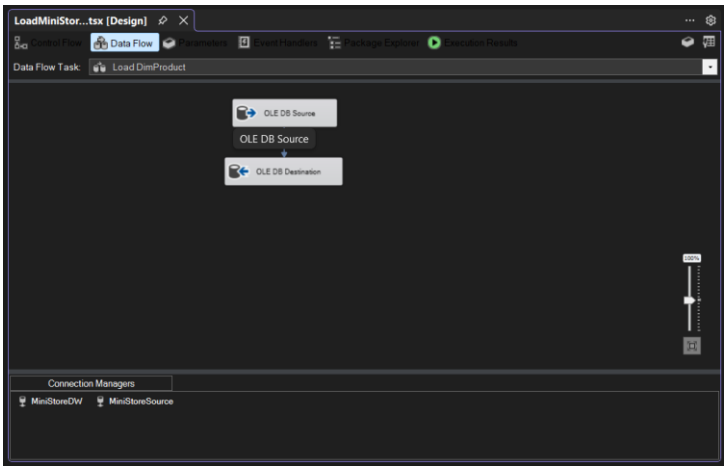


Figure 5: SSIS Data Flow (Load DimProduct)

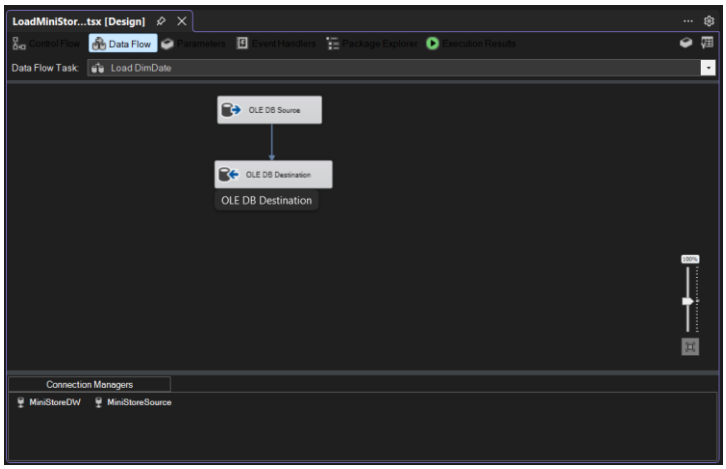


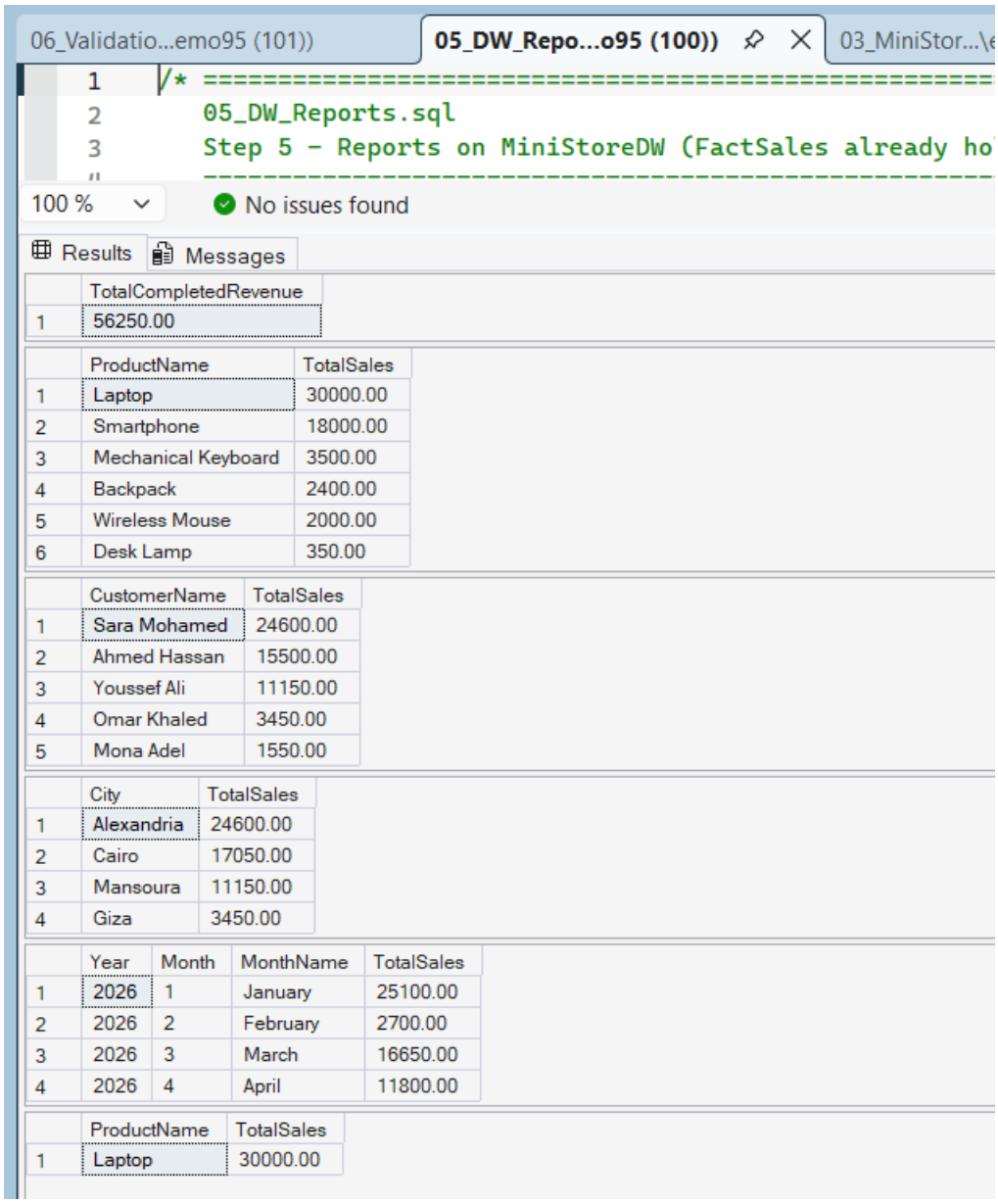
Figure 6: SSIS Data Flow (Load DimDate)

5. Data Warehouse Reports

The warehouse reporting queries use FactSales joined to the relevant dimensions. They provide the following business views:

- Total completed sales revenue across all fact rows.
- Sales amount by product, ordered from highest to lowest revenue.
- Sales amount by customer.
- Sales amount by city.
- Sales amount by year and month, ordered chronologically.
- The product or products with the highest total revenue.

Because cancelled order lines are not loaded into FactSales, these warehouse reports analyze completed sales only. The reports use SUM and GROUP BY, with joins to dimension tables to provide readable product, customer, city, and date attributes.



06_Validatio...emo95 (101)) | 05_DW_Repo...o95 (100)) | 03_MiniStor...\e

```
1  /* ================================================
2     05_DW_Reports.sql
3     Step 5 - Reports on MiniStoreDW (FactSales already ho
   ------------------------------------------------------
```

100 % — No issues found

Results | Messages

	TotalCompletedRevenue
1	56250.00

	ProductName	TotalSales
1	Laptop	30000.00
2	Smartphone	18000.00
3	Mechanical Keyboard	3500.00
4	Backpack	2400.00
5	Wireless Mouse	2000.00
6	Desk Lamp	350.00

	CustomerName	TotalSales
1	Sara Mohamed	24600.00
2	Ahmed Hassan	15500.00
3	Youssef Ali	11150.00
4	Omar Khaled	3450.00
5	Mona Adel	1550.00

	City	TotalSales
1	Alexandria	24600.00
2	Cairo	17050.00
3	Mansoura	11150.00
4	Giza	3450.00

	Year	Month	MonthName	TotalSales
1	2026	1	January	25100.00
2	2026	2	February	2700.00
3	2026	3	March	16650.00
4	2026	4	April	11800.00

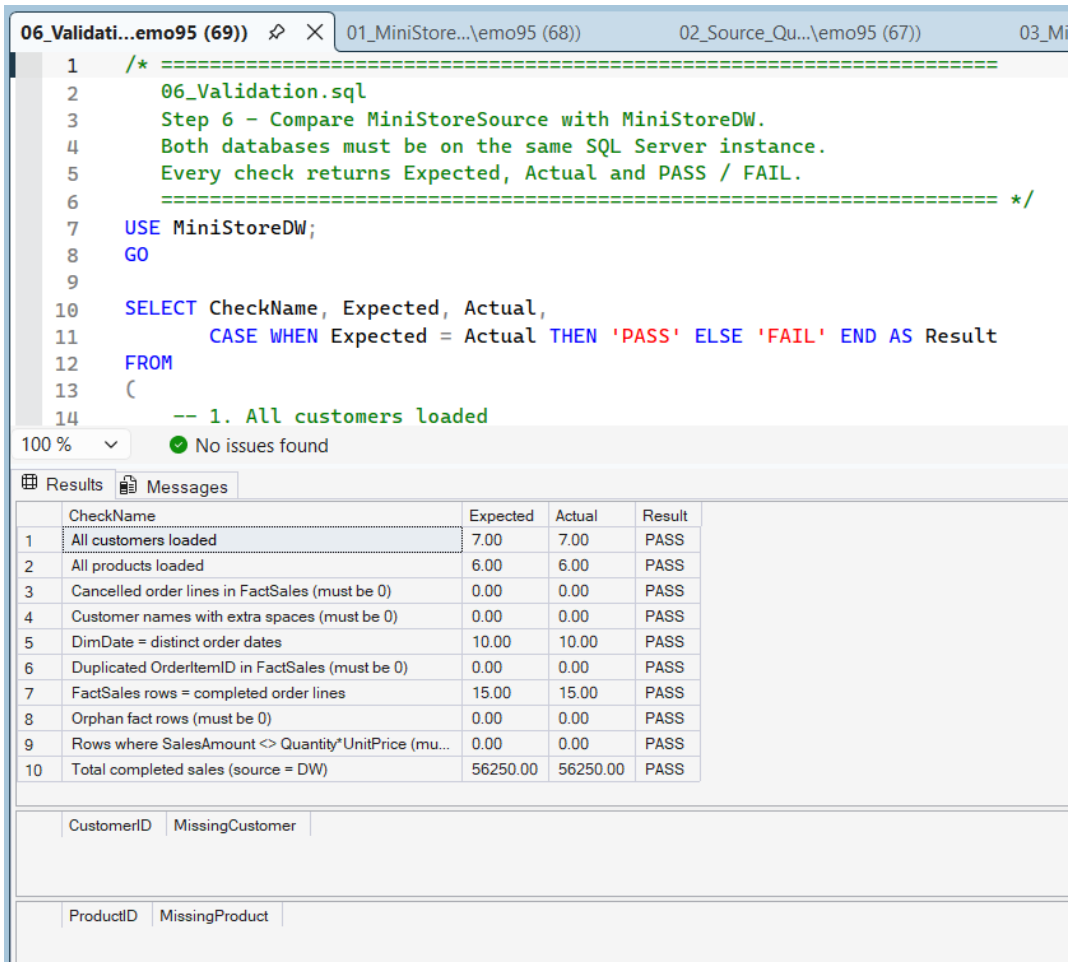
	ProductName	TotalSales
1	Laptop	30000.00

6. Data Validation

The validation script compares MiniStoreSource with MiniStoreDW and returns an Expected value, an Actual value, and a PASS/FAIL result for each check. The checks are designed to confirm:

- The customer and product dimension row counts match their source tables.
- DimDate contains one row for each distinct order date in the source.
- FactSales row count matches the number of order lines belonging to Completed orders.
- No cancelled or otherwise non-completed order line appears in FactSales.
- Every fact row references an existing customer, product, and date dimension record.
- SalesAmount equals Quantity multiplied by UnitPrice for every fact row.
- Total completed sales revenue in the source matches the total in FactSales.
- Customer names are trimmed and source order-line identifiers are not duplicated.

The validation checks should be executed after the SSIS package finishes. The final report should include the actual query output and should state that validation passed only if the relevant checks return PASS and the zero-exception checks return an actual value of 0.



06_Validati...emo95 (69)) | 01_MiniStore...\emo95 (68)) | 02_Source_Qu...\emo95 (67)) | 03_Mi

```
/* ====================================================================
   06_Validation.sql
   Step 6 - Compare MiniStoreSource with MiniStoreDW.
   Both databases must be on the same SQL Server instance.
   Every check returns Expected, Actual and PASS / FAIL.
   ==================================================================== */
USE MiniStoreDW;
GO

SELECT CheckName, Expected, Actual,
       CASE WHEN Expected = Actual THEN 'PASS' ELSE 'FAIL' END AS Result
FROM
(
    -- 1. All customers loaded
```

100 % | No issues found

Results | Messages

	CheckName	Expected	Actual	Result
1	All customers loaded	7.00	7.00	PASS
2	All products loaded	6.00	6.00	PASS
3	Cancelled order lines in FactSales (must be 0)	0.00	0.00	PASS
4	Customer names with extra spaces (must be 0)	0.00	0.00	PASS
5	DimDate = distinct order dates	10.00	10.00	PASS
6	Duplicated OrderItemID in FactSales (must be 0)	0.00	0.00	PASS
7	FactSales rows = completed order lines	15.00	15.00	PASS
8	Orphan fact rows (must be 0)	0.00	0.00	PASS
9	Rows where SalesAmount <> Quantity*UnitPrice (mu...	0.00	0.00	PASS
10	Total completed sales (source = DW)	56250.00	56250.00	PASS

	CustomerID	MissingCustomer

	ProductID	MissingProduct

7. Conclusion

This project demonstrates a basic end-to-end data engineering pipeline, from a relational operational database through SSIS-based ETL into a dimensional data warehouse, followed by SQL reporting and data-quality validation. The star schema supports sales analysis by product, customer, city, and month, while the completed-order rule keeps cancelled orders out of the sales fact table.

The final implementation evidence consists of the SQL scripts, the SSIS project and package, screenshots of successful execution, report query outputs, and validation results.